

Databases

Project : Designing a Telemetry and UX Database for *Chocolate-Doom*

Fergie Lindsay Díaz, Juan Esteban Ortega, Juan Esteban, Federico Mejia

today

Contents

1	Introduction and context	2
2	Assumptions and requirements	2
2.1	Assumptions	2
2.2	Functional requirements	2
2.3	Non-functional requirements	3
3	Survey Election - GUESS	3
3.1	What is it?	3
3.2	Why did we chose it?	4
4	Entity-Relation Diagram	4
5	Relational Schema - Initial Draft	5
5.1	Entities	5
5.2	Foreign Keys	5
5.3	Data Dictionary	5
6	Data Definition Language (DDL)	8
6.1	SQL Dictionary	8
6.2	1. AppUser	8
6.3	2. Player	9
6.4	3. Episode	9
6.5	4. Map	9
6.6	5. Sector	9
6.7	6. Game	9
6.8	7. GameParticipant	9
6.9	8. TelemetryEvent	9
6.10	9. UXInstrument	10
6.11	10. UXSubscale	10
6.12	11. UXItem	10
6.13	12. UXResponse	10
6.14	13. UXResponseItem	11
7	Conclusions	11

1 Introduction and context

This project is about designing and building a database to store and analyze data collected from a modified version of the game *Chocolate-Doom*. The game records detailed information during gameplay, such as the player's position, movement, and status at every moment. In addition to this, players also provide basic personal information and complete a user experience survey.

The main goal in the present report or document of this project is to organize all this data in a structured way so it can be stored, accessed, and analyzed efficiently. To do this, we will design a relational database that connects gameplay data with user and survey information. The database should be able to handle a large amount of data and ensure that everything is consistent and well organized.

Another important part of the project is to run queries that help us understand player behavior. For example, we will look at how players move, how close they are to each other, and whether they cooperate during the game. We will also relate this information to their survey responses to see how their experience connects to their behavior.

Finally, we will follow good practices for data handling, including cleaning and loading the data properly, and considering basic privacy and ethical aspects. Overall, the project aims to combine database design with practical analysis of real gameplay data.

2 Assumptions and requirements

2.1 Assumptions

- Each **User** is a real participant who has accepted consent before any data is collected.
- A **User** can have one or more **Player** identities (nicknames), but each **Player** belongs to only one **User**.
- Each **Game** represents one gameplay session with a start and end time.
- Telemetry data is recorded at every tic (very frequent events) for each player.
- Each telemetry record is uniquely identified by (game_id, player_id, tic).
- Each **Game** is linked to exactly one **Map** and one **Episode**.
- **Maps** are divided into **Sectors**, which help us analyze player positions.
- Each user answers only one UX survey after playing.
- Survey responses are completed after the game session and linked to the user.

2.2 Functional requirements

This next items will show what we assumed from the text, of what the system will need to do:

- Store and manage user demographic information (age, gender, experience level, consent).
- Support multiple player identities per user.
- Record high-frequency gameplay telemetry including position, momentum, orientation, and combat stats.
- Store game session metadata including episode, map, start time, and end time.
- Support spatial grouping of telemetry data by map sectors.
- Ingest raw telemetry logs (TSV format) into a structured relational database.

- Store and manage UX survey instruments (PENS, GUESS, BANGS).
- Store user responses to UX instruments at the question level.
- Support analytical queries on movement, proximity, and player behavior.
- Support aggregation of telemetry data for trajectory and cooperation analysis.

2.3 Non-functional requirements

- The database should handle large telemetry datasets efficiently (at least 20,000 records, but ideally much more as data grows).
- The schema should work well as the number of players, games, and telemetry records increases, without needing major changes.
- Primary keys and foreign keys should be used to keep data consistent across all tables.
- Indexes should be added to improve performance for common queries, especially those involving time sequences and spatial data.
- The design should allow new telemetry fields or new UX instruments to be added without redesigning the whole database.
- Data loading should be repeatable using a clear ETL process so the database can be rebuilt from raw logs if needed.
- User data should be anonymized, and consent must be stored before any data is collected or used.
- If some telemetry records are invalid or broken, they should be logged and skipped without stopping the whole ingestion process.

3 Survey Election - GUESS

The survey we decided to work with and base our design on is Game User-Experience Satisfaction Scale [GUESS](#).

3.1 What is it?

The Game User Experience Satisfaction Scale (GUESS-18) is a survey used to understand how players feel about a video game. It has 18 questions that are divided into nine sections, and each one looks at a different part of the experience, like how easy the game is to use, how fun it is, how immersive it feels, or how good the visuals and audio are.

Each question is answered using a scale from 1 to 7, where players say how much they agree or disagree. At the end, the answers are grouped to get scores for each section and also a total score that shows the overall satisfaction. There is one question that is reverse scored (“I feel bored while playing the game”), so it needs to be handled a bit differently.

GUESS-18 is actually a shorter version of a longer survey with 55 questions, but this one is faster to complete and still gives good results.

3.2 Why did we chose it?

We chose GUESS-18 because it is easy to use and still gives a lot of useful information. It lets us measure different aspects of the player experience, not just if the game works, but also if it is fun, engaging, or interesting.

Another reason is that it is a well-known and tested survey, so we can trust the results we get. This is important because we want to compare what players say with the data we collect from the game.

It also fits nicely with our database design. Since the survey is divided into sections and questions, we can store everything in an organized way and later analyze things like enjoyment or immersion separately.

Finally, it is short, so players can answer it quickly after playing. This makes it more likely that we get complete and honest responses.

4 Entity-Relation Diagram

This section presents the initial conceptual model of the system. It defines the main entities involved in the telemetry and UX data pipeline and illustrates how they interact. The ER diagram serves as the foundation for the relational schema that will be implemented in later stages of the project.

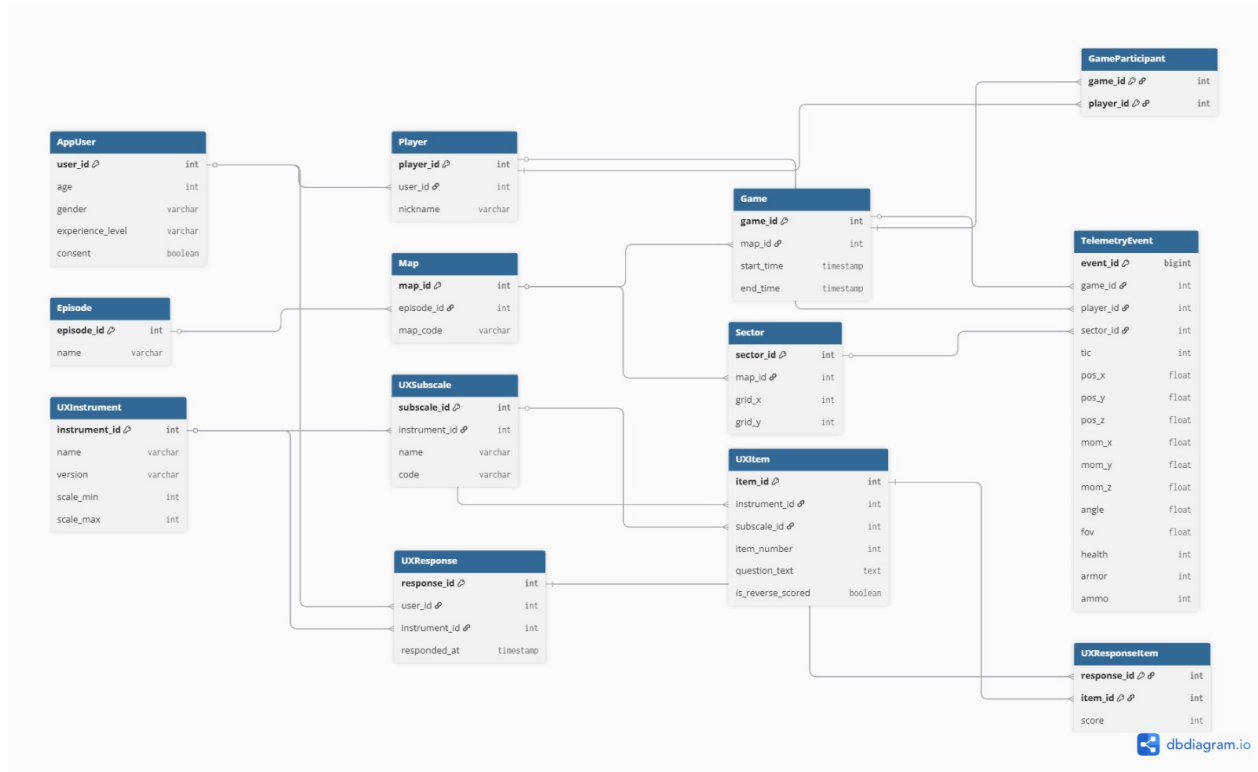


Figure 1: Entity-Relation Diagram

The diagram organizes gameplay telemetry, user information, and survey responses into coherent relationships. It highlights how users are linked to players, how players participate in game sessions, and how each session generates detailed telemetry data. It also integrates UX instruments to connect behavioral data with subjective user experience.

5 Relational Schema - Initial Draft

This section translates the conceptual ER model into a relational schema. It defines the entities, entities with attributes, and keys required to implement the database in a structured and normalized form.

5.1 Entities

This subsection lists the core entities identified in the system. These represent the main objects stored in the database, including users, gameplay sessions, telemetry records, and UX survey components.

- AppUser
- Player
- Game
- GameParticipant
- TelemetryEvent
- Episode
- Map
- Sector
- UXInstrument
- UXItem
- UXResponse
- UXResponseItem

5.2 Foreign Keys

The way in which we write or show the foreign keys will later help us in the way we can use it in SQL.

Example:

```
FOREIGN KEY (user_id) REFERENCES User(user_id)
```

This subsection defines the relationships between tables using foreign keys. These constraints enforce referential integrity and ensure consistency across users, gameplay sessions, telemetry events, and UX responses.

5.3 Data Dictionary

AppUser

Attribute	Type	Constraints	Description
user_id	SERIAL	PK	Unique identifier for each user
age	INTEGER	CHECK (10–100)	Age of the participant
gender	VARCHAR(50)	—	Self-reported gender
experience_level	VARCHAR(50)	—	Level of gaming experience
consent	BOOLEAN	NOT NULL, DEFAULT FALSE	Indicates informed consent for participation

Player

player_id	SERIAL	PK	Unique identifier for each in-game player
user_id	INTEGER	FK, NOT NULL	References AppUser(user_id); links player to a user
nickname	VARCHAR(100)	NOT NULL	Player's in-game alias

Episode

episode_id	SERIAL	PK	Unique episode identifier
name	VARCHAR(100)	NOT NULL	Name of the episode

Map

map_id	SERIAL	PK	Unique map identifier
episode_id	INTEGER	FK	References Episode(episode_id)
map_code	VARCHAR(50)	NOT NULL	Map code (e.g., E1M1)

Sector

sector_id	SERIAL	PK	Unique sector identifier
map_id	INTEGER	FK	References Map(map_id)
grid_x	INTEGER	—	X-coordinate within the map grid
grid_y	INTEGER	—	Y-coordinate within the map grid

Game

game_id	SERIAL	PK	Unique identifier for each game session
map_id	INTEGER	FK	References Map(map_id)
start_time	TIMESTAMP	NOT NULL	Timestamp when the session started
end_time	TIMESTAMP	—	Timestamp when the session ended

GameParticipant

game_id	INTEGER	PK, FK	References Game(game_id)
player_id	INTEGER	PK, FK	References Player(player_id)

TelemetryEvent

event_id	BIGSERIAL	PK	Unique identifier for each telemetry record
game_id	INTEGER	FK	References Game(game_id)
player_id	INTEGER	FK	References Player(player_id)
sector_id	INTEGER	FK	Nullable reference to Sector(sector_id)
tic	INTEGER	NOT NULL	Discrete time unit (game tick)
pos_x	DOUBLE PRECISION	NOT NULL	Player position on X-axis
pos_y	DOUBLE PRECISION	NOT NULL	Player position on Y-axis
pos_z	DOUBLE PRECISION	NOT NULL	Player position on Z-axis
mom_x	DOUBLE PRECISION	—	Momentum on X-axis
mom_y	DOUBLE PRECISION	—	Momentum on Y-axis
mom_z	DOUBLE PRECISION	—	Momentum on Z-axis
angle	DOUBLE PRECISION	—	Player orientation angle
fov	DOUBLE PRECISION	—	Field of view
health	INTEGER	—	Player health value
armor	INTEGER	—	Player armor value
ammo	INTEGER	—	Ammunition count

Constraint: UNIQUE (game_id, player_id, tic)

UXInstrument

instrument_id	SERIAL	PK	Unique identifier for each UX instrument
name	VARCHAR(100)	NOT NULL	Name of the instrument (e.g., GUESS, PENS)
version	VARCHAR(50)	—	Instrument version
scale_min	INTEGER	—	Minimum possible score
scale_max	INTEGER	—	Maximum possible score

UXSubscale

subscale_id	SERIAL	PK	Unique identifier for each subscale
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
name	VARCHAR(100)	NOT NULL	Name of the subscale
code	VARCHAR(20)	—	Short code identifier

UXItem

item_id	SERIAL	PK	Unique identifier for each item/question
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
subscale_id	INTEGER	FK	References UXSubscale(subscale_id)
item_number	INTEGER	NOT NULL	Item order within the instrument
question_text	TEXT	NOT NULL	Text of the survey question
is_reverse_scored	BOOLEAN	DEFAULT FALSE	Indicates if reverse scoring is applied

UXResponse

response_id	SERIAL	PK	Unique identifier for each survey response
user_id	INTEGER	FK	References AppUser(user_id)
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
responded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of submission

UXResponseItem

response_id	INTEGER	PK, FK	References UXResponse(response_id)
item_id	INTEGER	PK, FK	References UXItem(item_id)
score	INTEGER	CHECK (≥ 0)	Score given to the item

6 Data Definition Language (DDL)

6.1 SQL Dictionary

- **SERIAL - BIGSERIAL**: Auto-increment numbers for IDs. *Example*: In AppUser, user_id SERIAL PRIMARY KEY gives each user a unique ID. In TelemetryEvent, event_id BIGSERIAL PRIMARY KEY allows billions of events.
- **CHECK**: Rule that makes sure values are valid. *Example*: In AppUser, age INTEGER CHECK (age BETWEEN 10 AND 100) only allows ages between 10 and 100. In UXResponseItem, score INTEGER CHECK (score ≥ 0) ensures scores are not negative.
- **ON DELETE CASCADE**: If a parent row is deleted, linked child rows are deleted too. *Example*: If an AppUser is deleted, their Player record is also deleted. If a Game is deleted, all its GameParticipants are removed.
- **TIMESTAMP**: Stores both date and time. *Example*: In Game, start_time TIMESTAMP NOT NULL records when the game started. In UXResponse, responded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP saves the time a user answered.
- **PRECISION**: Controls how exact numbers are stored. *Example*: In TelemetryEvent, pos_x DOUBLE PRECISION stores very detailed player positions, momentum, and angles.
- **ON DELETE SET NULL**: If a parent row is deleted, the link is removed but the child row stays (foreign key becomes NULL). *Example*: If a Sector is deleted, the TelemetryEvent still exists but its sector_id becomes NULL. If a UXSubscale is deleted, the UXItem remains but loses its subscale link.

6.2 1. AppUser

```
CREATE TABLE AppUser (  
    user_id SERIAL PRIMARY KEY,  
    age INTEGER CHECK (age BETWEEN 10 AND 100),  
    gender VARCHAR(50),  
    experience_level VARCHAR(50),  
    consent BOOLEAN NOT NULL DEFAULT FALSE  
);
```

6.3 2. Player

```
CREATE TABLE Player (  
    player_id SERIAL PRIMARY KEY,  
    user_id INTEGER NOT NULL REFERENCES AppUser(user_id) ON DELETE CASCADE,  
    nickname VARCHAR(100) NOT NULL  
);
```

6.4 3. Episode

```
CREATE TABLE Episode (  
    episode_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL  
);
```

6.5 4. Map

```
CREATE TABLE Map (  
    map_id SERIAL PRIMARY KEY,  
    episode_id INTEGER REFERENCES Episode(episode_id) ON DELETE CASCADE,  
    map_code VARCHAR(50) NOT NULL  
);
```

6.6 5. Sector

```
CREATE TABLE Sector (  
    sector_id SERIAL PRIMARY KEY,  
    map_id INTEGER REFERENCES Map(map_id) ON DELETE CASCADE,  
    grid_x INTEGER,  
    grid_y INTEGER  
);
```

6.7 6. Game

```
CREATE TABLE Game (  
    game_id SERIAL PRIMARY KEY,  
    map_id INTEGER REFERENCES Map(map_id) ON DELETE CASCADE,  
    start_time TIMESTAMP NOT NULL,  
    end_time TIMESTAMP  
);
```

6.8 7. GameParticipant

```
CREATE TABLE GameParticipant (  
    game_id INTEGER REFERENCES Game(game_id) ON DELETE CASCADE,  
    player_id INTEGER REFERENCES Player(player_id) ON DELETE CASCADE,  
    PRIMARY KEY (game_id, player_id)  
);
```

6.9 8. TelemetryEvent

```
CREATE TABLE TelemetryEvent (  
    event_id BIGSERIAL PRIMARY KEY,  
    game_id INTEGER REFERENCES Game(game_id) ON DELETE CASCADE,
```

```

    player_id INTEGER REFERENCES Player(player_id) ON DELETE CASCADE,
    sector_id INTEGER REFERENCES Sector(sector_id) ON DELETE SET NULL,
    tic INTEGER NOT NULL,
    pos_x DOUBLE PRECISION NOT NULL,
    pos_y DOUBLE PRECISION NOT NULL,
    pos_z DOUBLE PRECISION NOT NULL,
    mom_x DOUBLE PRECISION,
    mom_y DOUBLE PRECISION,
    mom_z DOUBLE PRECISION,
    angle DOUBLE PRECISION,
    fov DOUBLE PRECISION,
    health INTEGER,
    armor INTEGER,
    ammo INTEGER,
    CONSTRAINT uq_telem UNIQUE (game_id, player_id, tic)
);

```

6.10 9. UXInstrument

```

CREATE TABLE UXInstrument (
    instrument_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    version VARCHAR(50),
    scale_min INTEGER,
    scale_max INTEGER
);

```

6.11 10. UXSubscale

```

CREATE TABLE UXSubscale (
    subscale_id SERIAL PRIMARY KEY,
    instrument_id INTEGER REFERENCES UXInstrument(instrument_id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    code VARCHAR(20)
);

```

6.12 11. UXItem

```

CREATE TABLE UXItem (
    item_id SERIAL PRIMARY KEY,
    instrument_id INTEGER REFERENCES UXInstrument(instrument_id) ON DELETE CASCADE,
    subscale_id INTEGER REFERENCES UXSubscale(subscale_id) ON DELETE SET NULL,
    item_number INTEGER NOT NULL,
    question_text TEXT NOT NULL,
    is_reverse_scored BOOLEAN DEFAULT FALSE
);

```

6.13 12. UXResponse

```

CREATE TABLE UXResponse (
    response_id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES AppUser(user_id) ON DELETE CASCADE,
    instrument_id INTEGER REFERENCES UXInstrument(instrument_id) ON DELETE CASCADE,
    responded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
);
```

6.14 13. UXResponseItem

```
CREATE TABLE UXResponseItem (  
    response_id INTEGER REFERENCES UXResponse(response_id) ON DELETE CASCADE,  
    item_id INTEGER REFERENCES UXItem(item_id) ON DELETE CASCADE,  
    score INTEGER NOT NULL CHECK (score >= 0),  
    PRIMARY KEY (response_id, item_id)  
);
```

7 Conclusions

In this project, we designed and implemented a relational database to store gameplay telemetry together with user information and UX survey data. The schema follows a structured approach that separates core entities such as users, games, telemetry events, and survey responses, making the system organized and scalable.

One important result is that we can connect player behavior with their subjective experience. By linking telemetry data with survey responses, we can analyze not only how players move in the game, but also how they feel about it. This allows more complete analysis, for example comparing engagement with movement patterns and cooperation between players.

We also considered how the data is loaded into the system. By using an ETL approach, we can take raw telemetry logs, clean them, and insert them into the database in a consistent way. This ensures the data is reliable and can be reused if we need to reload everything.

Additionally, the use of primary keys, foreign keys, and indexes helps maintain data integrity and improves performance when running analytical queries. This is important because telemetry data grows very fast and queries need to be efficient.

Overall, this project shows how a relational database can be used not only to store large amounts of data, but also to support meaningful analysis of gameplay behavior and user experience.